

手写与打印文字智能分割技术标注方法

目录

1 背景.....	3
2 文件说明.....	3
2.1 第一种标注方法.....	3
2.2 第二种标注方法.....	3
3 第一种标注方法：数据集统一标注方法.....	4
3.1 初始化路径.....	4
3.2 导入图片.....	6
3.3 重叠区域标注.....	8
3.4 擦除手写字.....	11
3.5 导出.....	13
3.6 图片二值化.....	17
3.7 背景擦除.....	18
3.8 生成掩码和重构图.....	20
4 第二种标注方法：测试集文字掩码标注方法.....	22

1 背景

为了提高模型的准确性以及有效地检验模型准确率，本文指出两种标注方法：第一种标注方法用于制作标准的数据集，用于模型的训练、验证和测试阶段；第二种标注方法以字为单位进行检验，用于算法交付阶段测试使用。所用标注工具为开源软件 GIMP ([GIMP - Downloads](#)) (Photoshop 也能达到同样的效果)。

2 文件说明

2.1 第一种标注方法

在制作数据集时，用到了以下类型的图片：

images, 原图，由甲方扫描真实场景下的纸质文件获取
erased, 擦除手写字图，通过 Gimp 擦除原图上的手写字内容生成的图片
overlap_mask, 重叠区域掩码图，通过 Gimp 进行像素级别标注生成的图片
images_binary, 原图的二值图，通过脚本对原图二值化获得
erased_binary, 擦除手写字的二值图，通过脚本对擦除手写字图二值化获得
mask, 最终掩码，在获取上述图片以后，通过脚本生成的像素级别标注图
rebuild, GT 图，在获取上述图片以后，通过脚本生成的擦除手写字图（白色背景）

大概执行流程：

- a. 选择一个空白目录作为数据集标注的根目录，在 tools/dir_settings.py 设置路径
- b. 调用 init_dir.py 函数，在根目录下生成七个空路径
- c. 将原图复制到根目录下的 images 中
- d. 使用 Gimp 对重叠区域进行标注，生成 overlap_mask 的图片
- e. 使用 Gimp 擦除手写字区域，生成 erased 的图片
- f. 使用 single_binarize 二值化 images 图和 erased 图，分别生成 images_binary 图和 erased_binary 图
- g. 使用 Gimp 擦除掉 images_binary 图和 erased_binary 图中的背景和噪声
- h. 调用 batch_mask.py，生成 mask 图，调用 batch_rebuild.py，生成 rebuild 图

2.2 第二种标注方法

直接使用 Gimp 对单个字区域进行标注

3 第一种标注方法：数据集统一标注方法

对于所有字进行逐像素标注的人工成本过大，这里给出一套系统方案：对于重叠区域人工逐像素点进行标注，对于不重叠的手写字，用工具手动擦除，最后调用脚本给背景像素、打印体、手写体以及重叠部分给出标签，并且提供模拟擦除手写字之后的 ground truth。

3.1 初始化路径

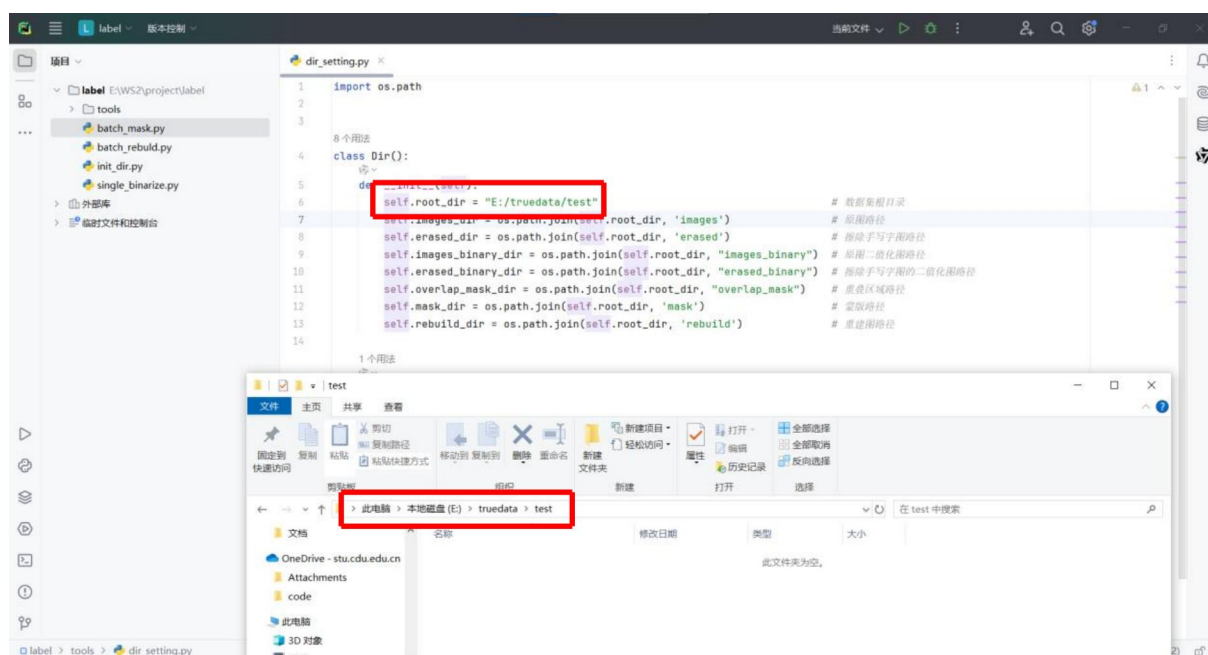


图1 初始化路径图

注意，在设置 `self.root_dir` 时，路径中应当只有正斜杠“/”。如果复制过来的路径出现了“\”，请全部手动替换成“/”。否则脚本将无法识别正确路径。

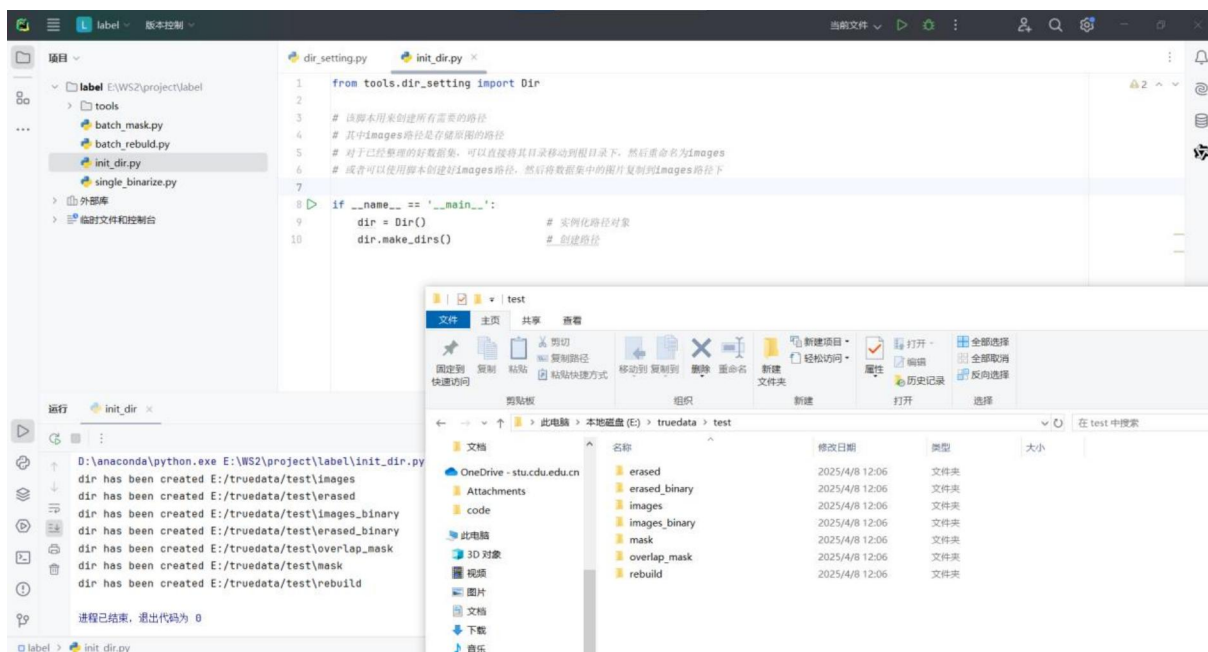


图2 初始化后的路径

使用脚本后，根目录下生成了多个空目录，用来存储不同类别的图片

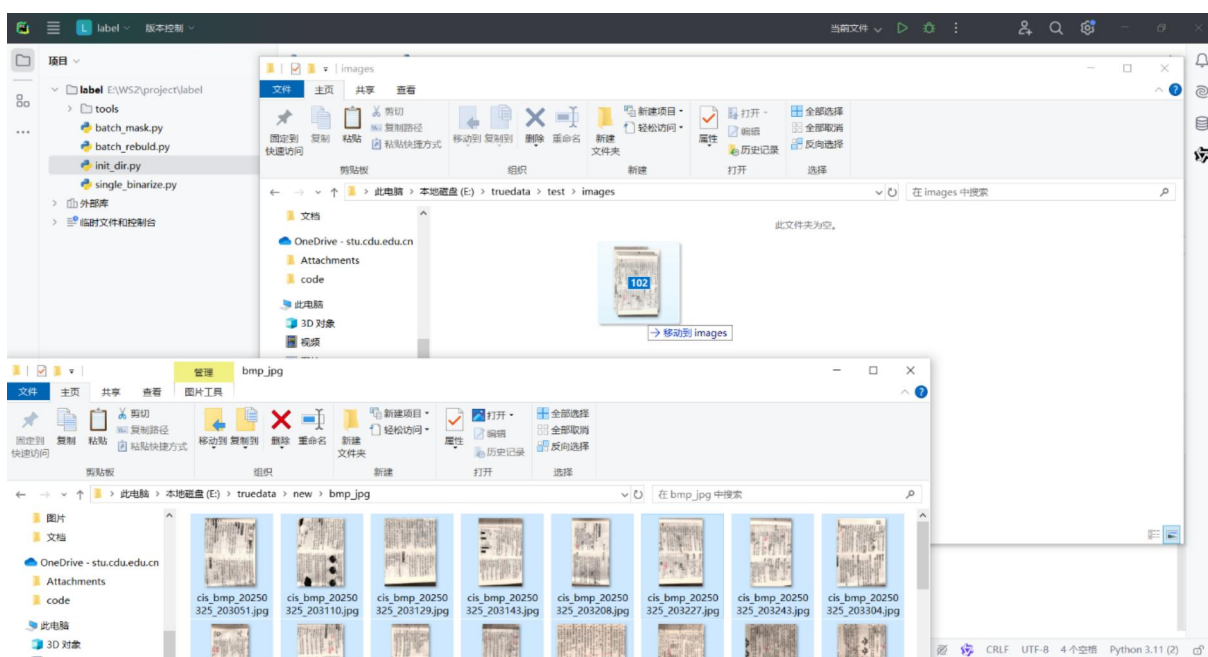


图3 移动数据集原图到 images

将扫描仪获取的原始图片移动到 images 目录下，便于后续制作数据集。本样例中移动一张图片作为示范、

3.2 导入图片

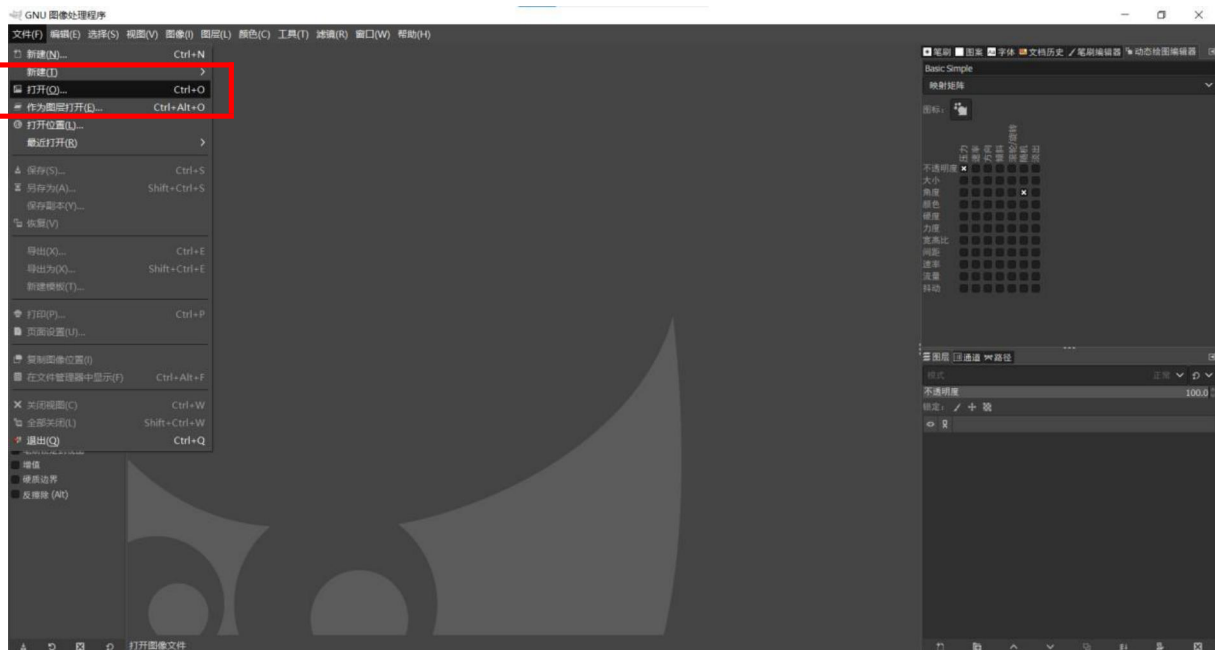


图4 导入图片 1

点击右上角，文件——>打开

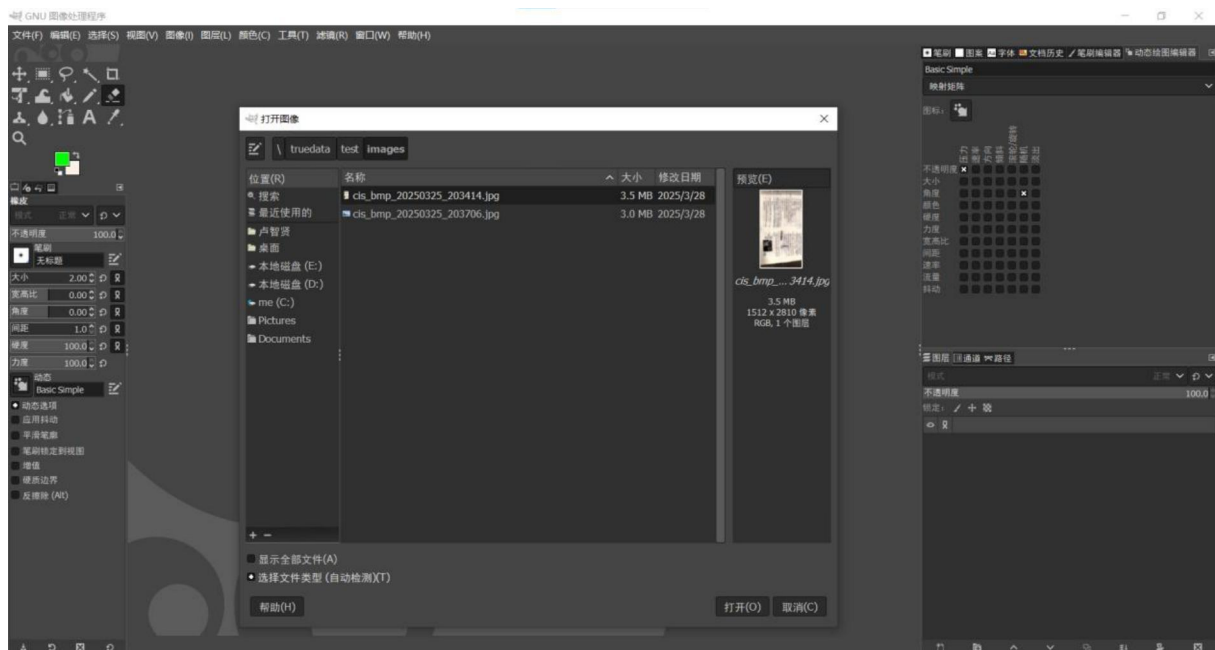


图5 导入图片 2

找到根目录下的 images 路径，打开一张图片。

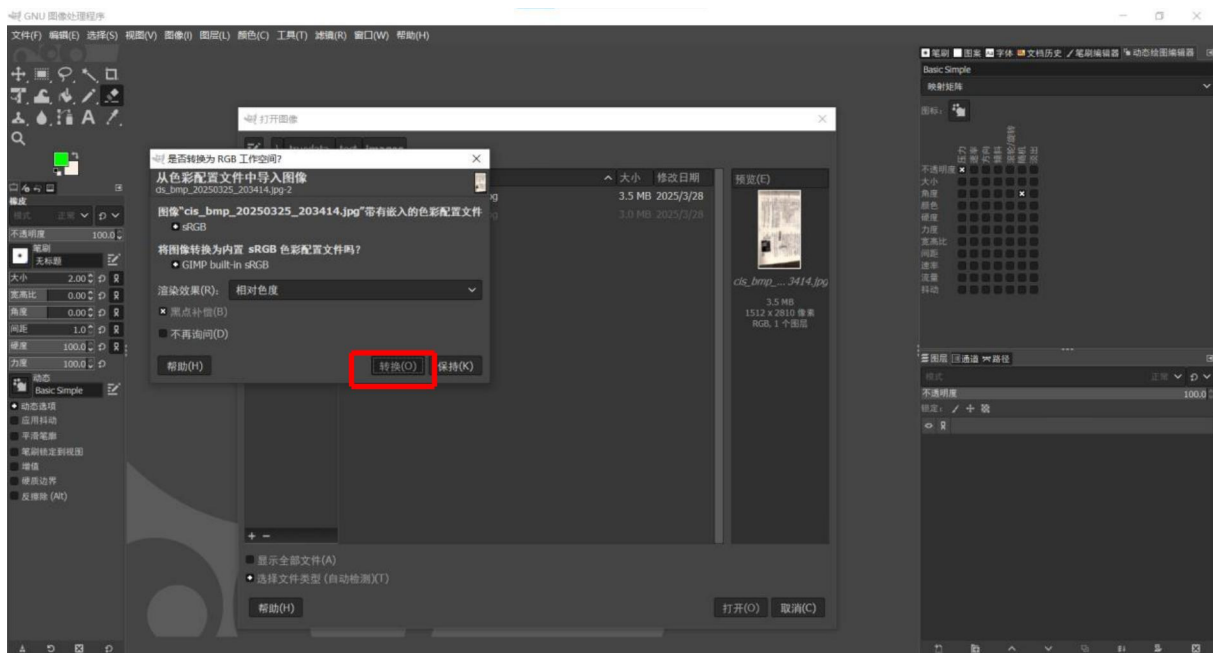


图6 导入图片 3

点击转换即可

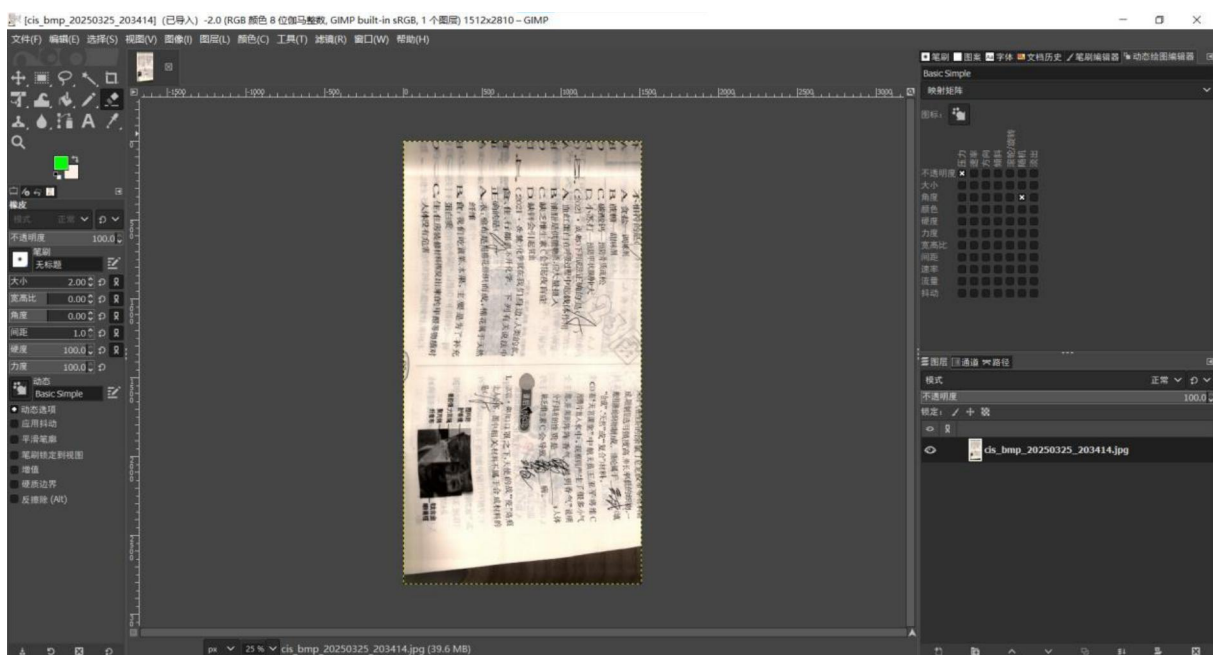


图7 导入图片 4

导入图片后的界面如图所示

3.3 重叠区域标注

为了提高模型分割的准确性，对于重叠部分，需要逐像素手动标注

新建图层，进行标注。重叠区域作为最难操作的部分，其几乎只能用肉眼判断重叠区域，并用画笔逐像素填充，因此需要投入充足耐心。

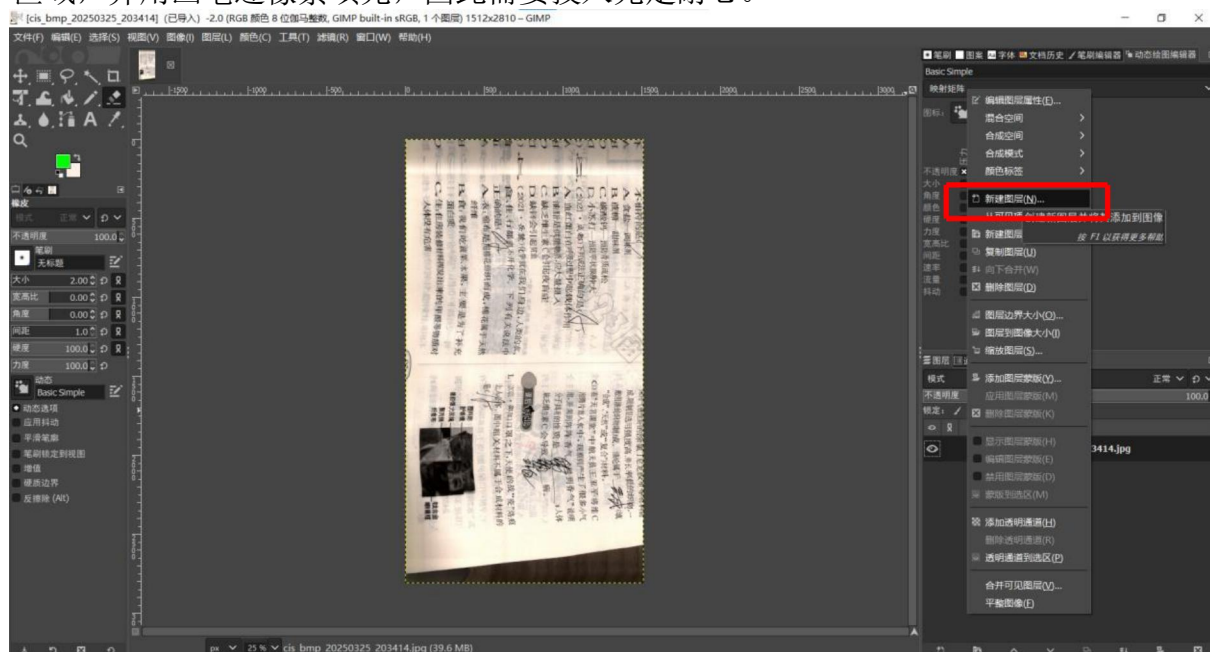


图8 新建图层 1

右击右下角空白区域，新建图层

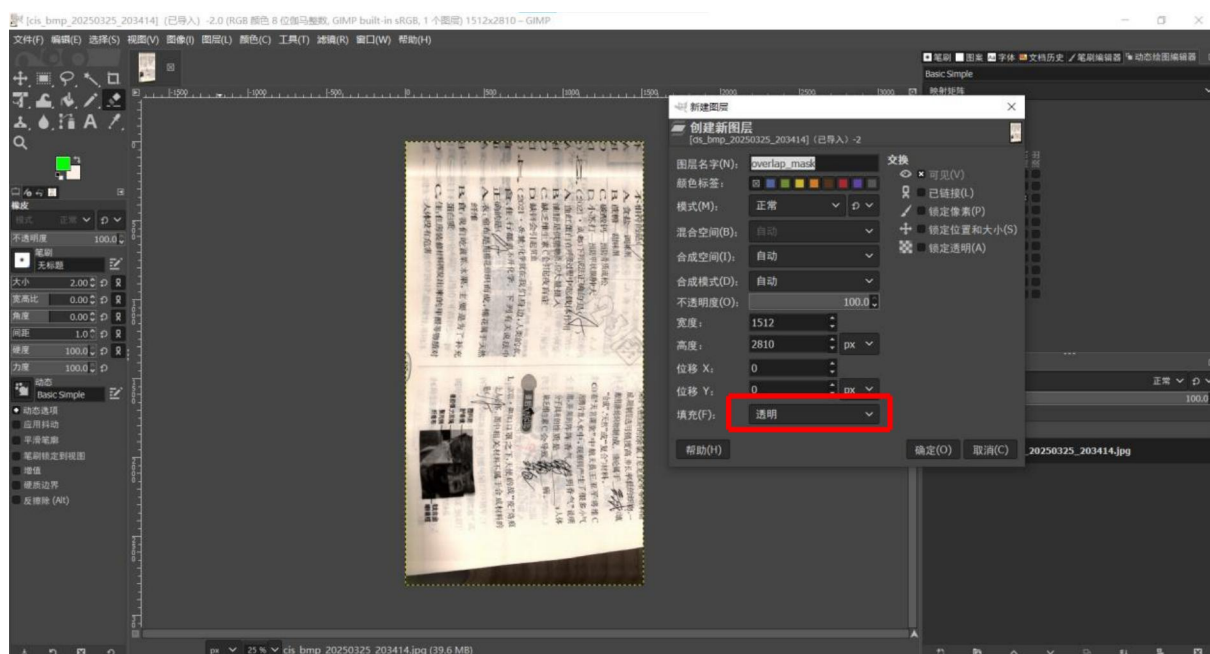


图9 新建图层 2

注意设置透明背景，方便后续标注

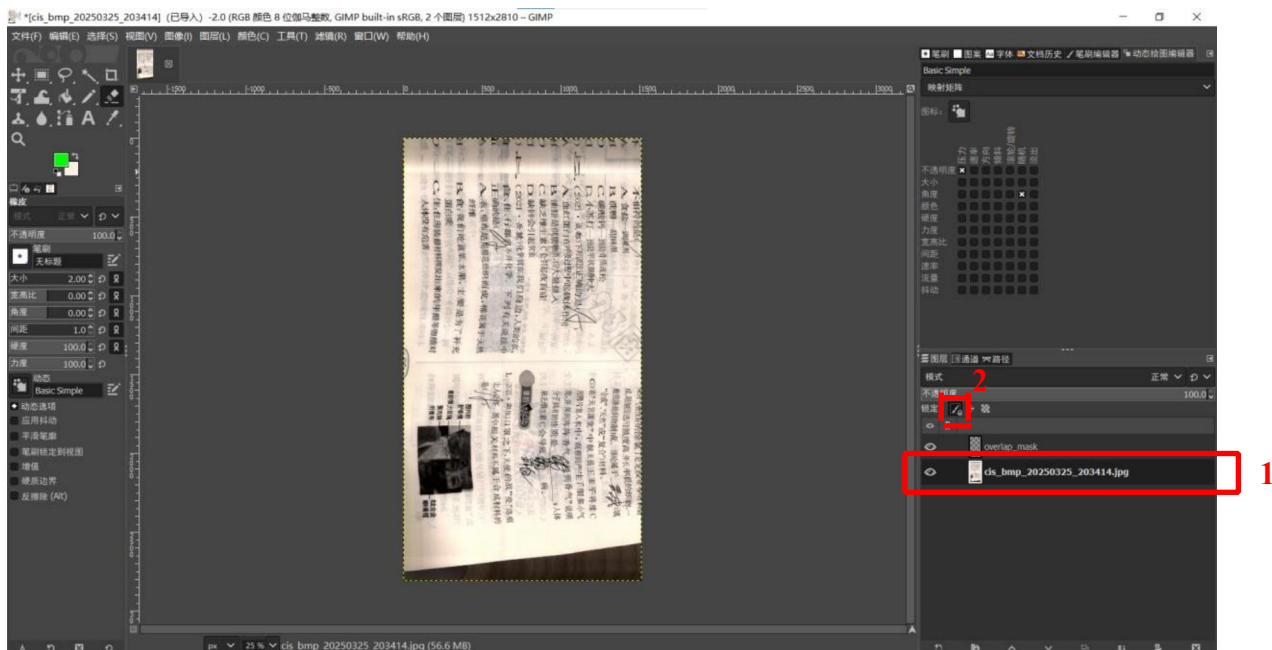


图10 锁定原图层像素

为了防止对原图的误修改，这里可以将原图像素锁定：首先选中原图层，点击画笔图标锁定原图层像素。

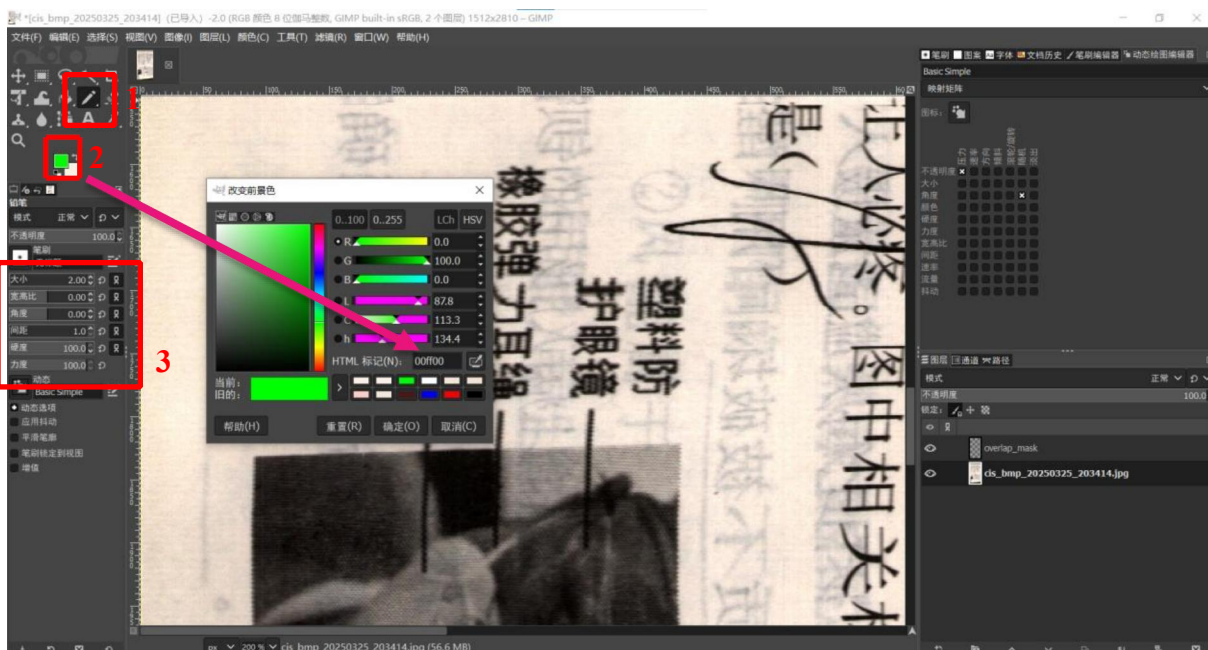


图11 标注设置

在像素标注时一定要选择新建的图层!!! 这里演示的选择 overlap_mask。

选取画笔，设置颜色为 00ff00（绿色），画笔大小不要过大（建议 2）。硬度和力量设置最大，防止颜色不均匀。选取新建图层，并且将画面放大到合适的大小。

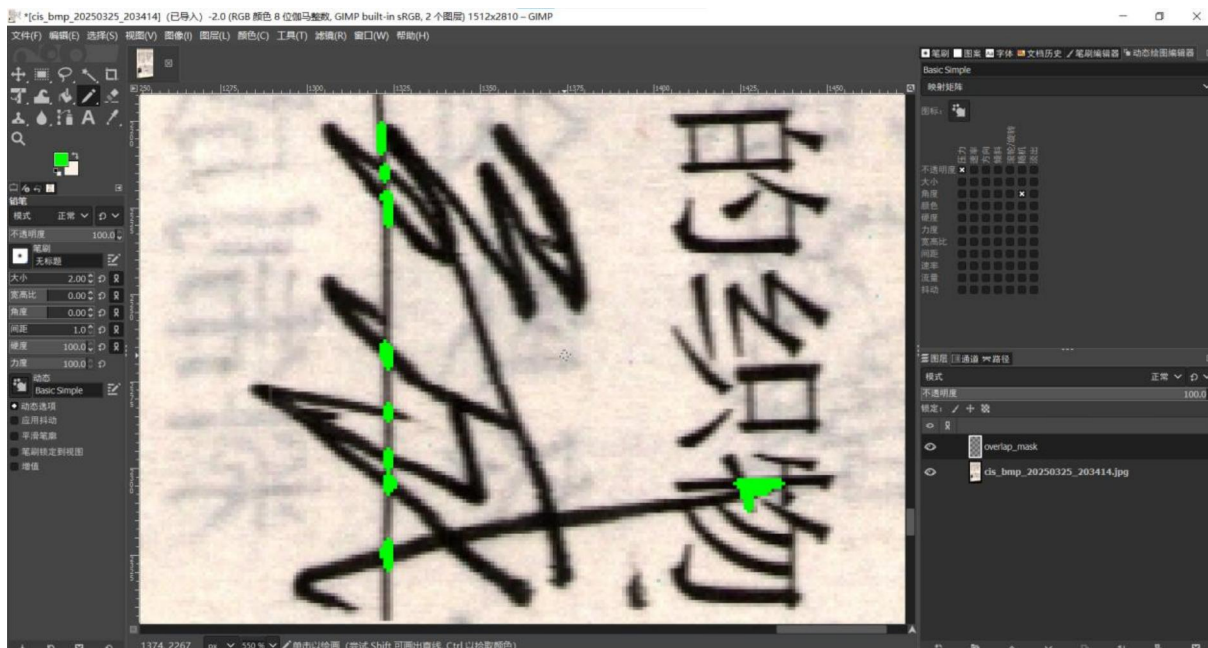


图12 重叠部分标注效果

可以取消左下角的眼睛图案，隐藏图层，检查标注是否到位。在重叠部分的打印字与手写字颜色一样时，要更加注意标注的细节。

整张图片所有的重叠部分标注完成时，将该图层锁定（即新建的，用来标注重叠部分的图层），防止后续误操作修改。

3.4 擦除手写字

端到端的训练中，我们需要提供擦除完手写字后的图片作为标签样本（GT，ground truth），因此需要用 GIMP 工具中的橡皮擦来实现擦除。

擦除的颜色选择与背景相似的颜色，点击选择原图，使用拾色器点击背景颜色。

为什么要用背景颜色擦除手写字，而不是直接用白色？因为后续使用二值化脚本时，如果自己无法设置一个很好的阈值，可以用自适应二值化函数，这时，颜色过白的背景会导致噪声影响变大，不利于处理。

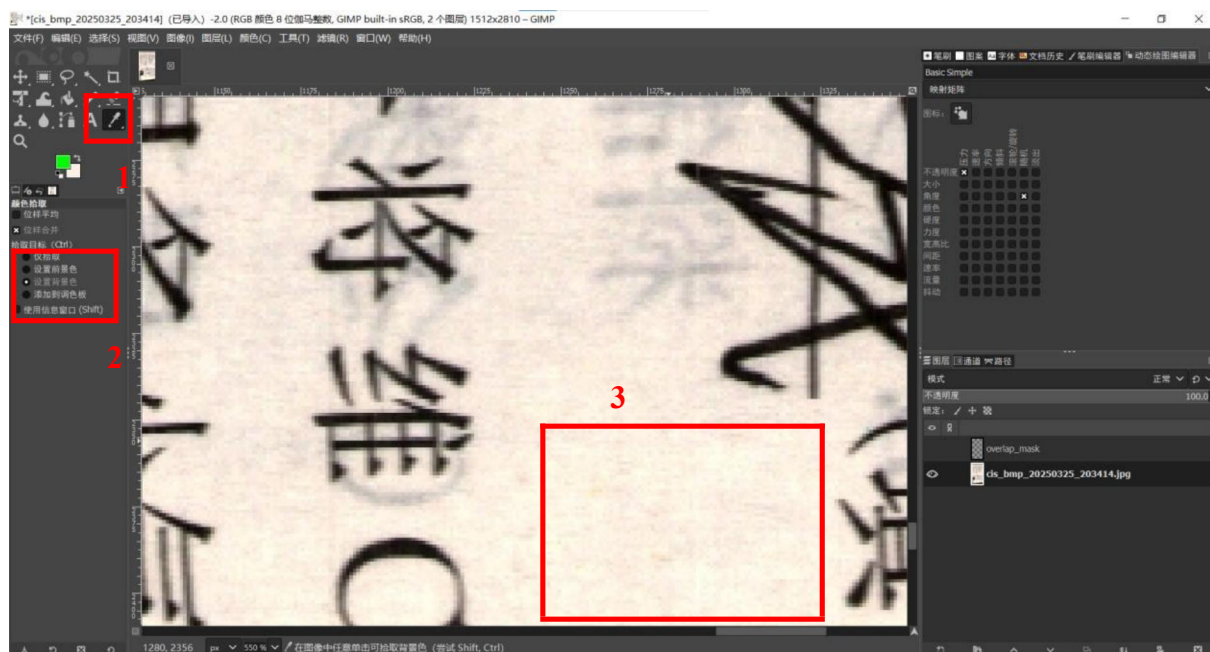


图13 工具设置



图14 橡皮擦设置

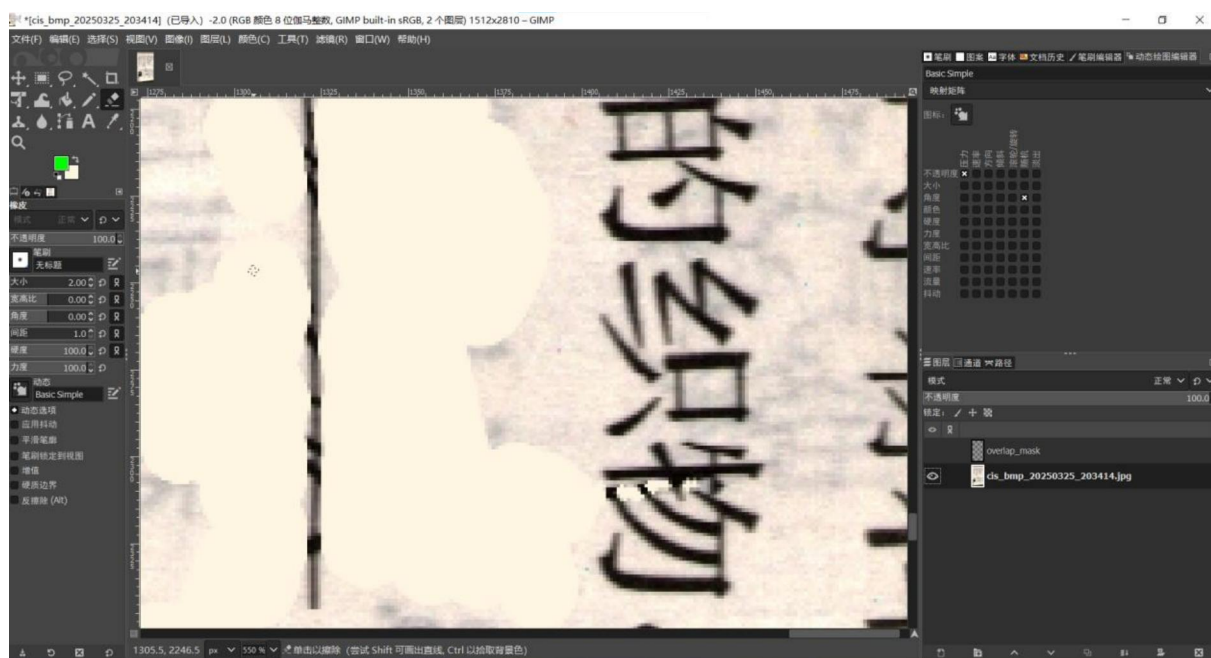


图15 擦除效果图

在擦除汉字重叠部分的边缘，将橡皮擦大小调小（2 左右），精细擦除手写字部分。

3.5 导出

到目前为止，需要导出两张图片，一张图片是重叠部分的掩码图片，一张图片是擦除手写字图片。导出图片时注意，如果导出手写字擦除图片，那么需要隐藏掩码图层；导出掩码图片，需要隐藏原图层。

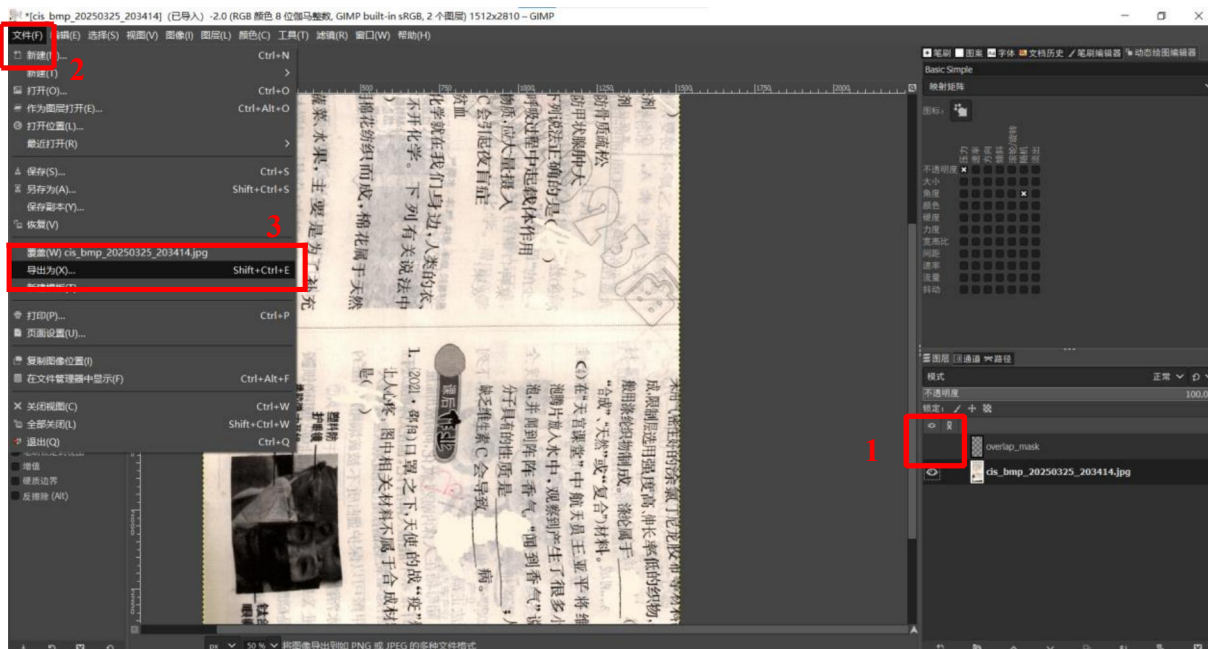


图16 导出手写字擦除图片 1

右上角 文件——>导出为

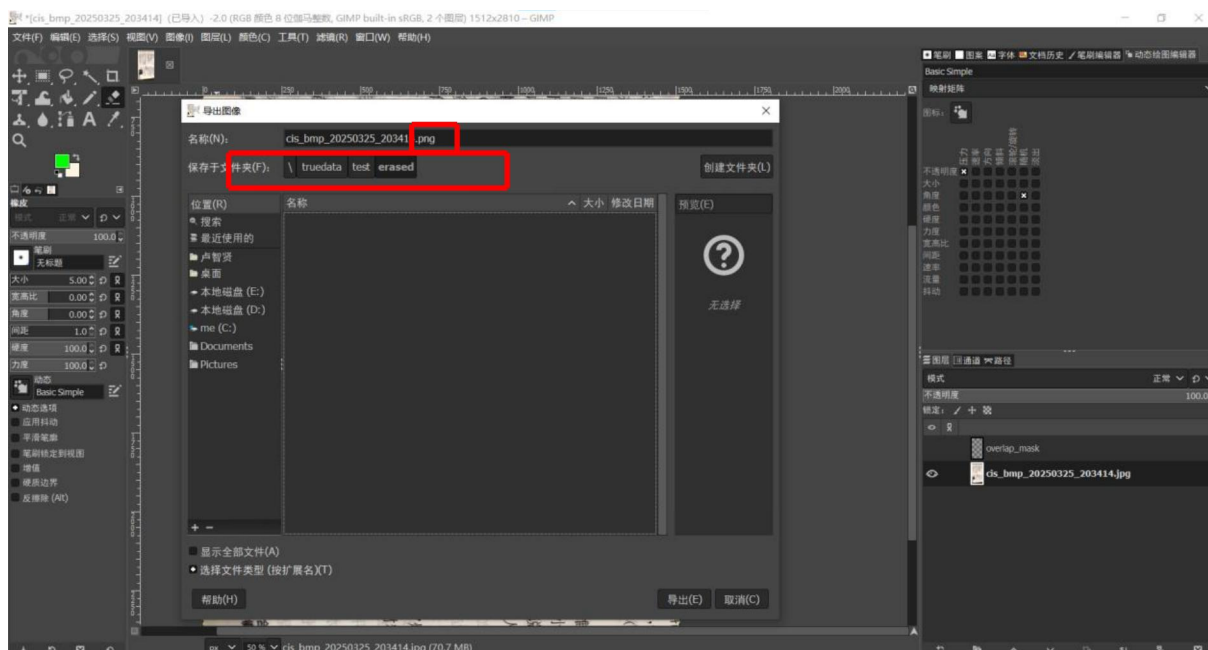


图17 导出手写字擦除图片 1

导出的名字必须要与读取的名字一致。并且要注意，导出的格式为png，导出的目录为根目录下的erased

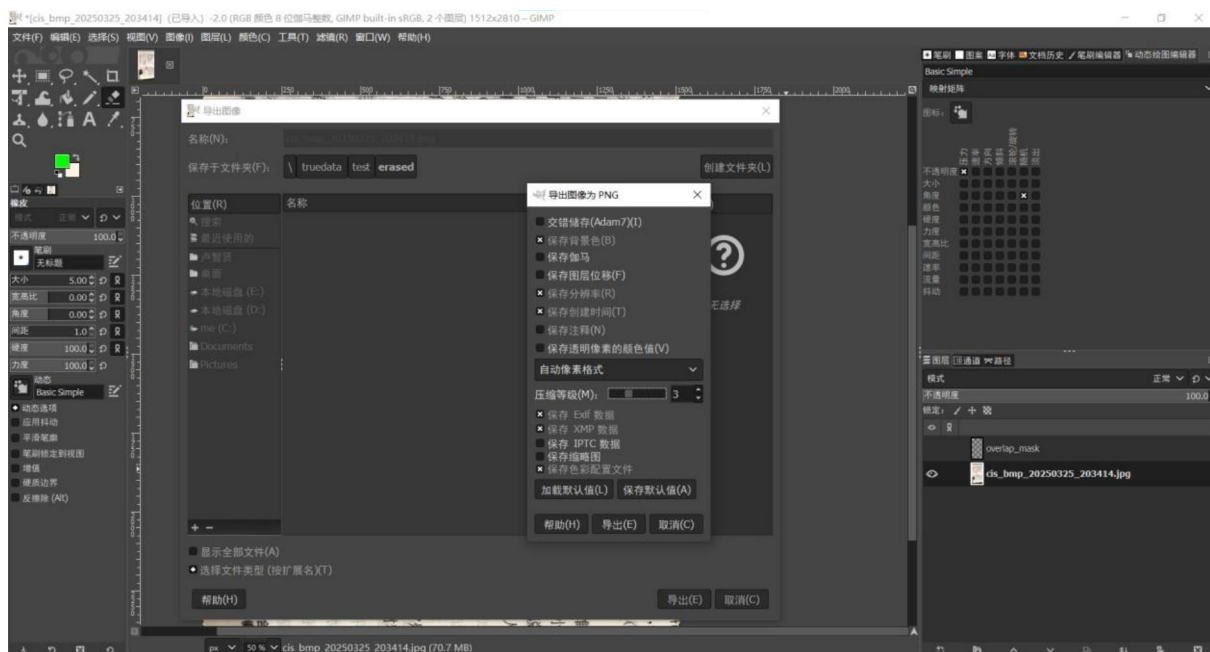


图18 导出手写字擦除图片 1

压缩等级选择 3，使用 Ps 工具可能没有压缩等级选项，直接保存即可。

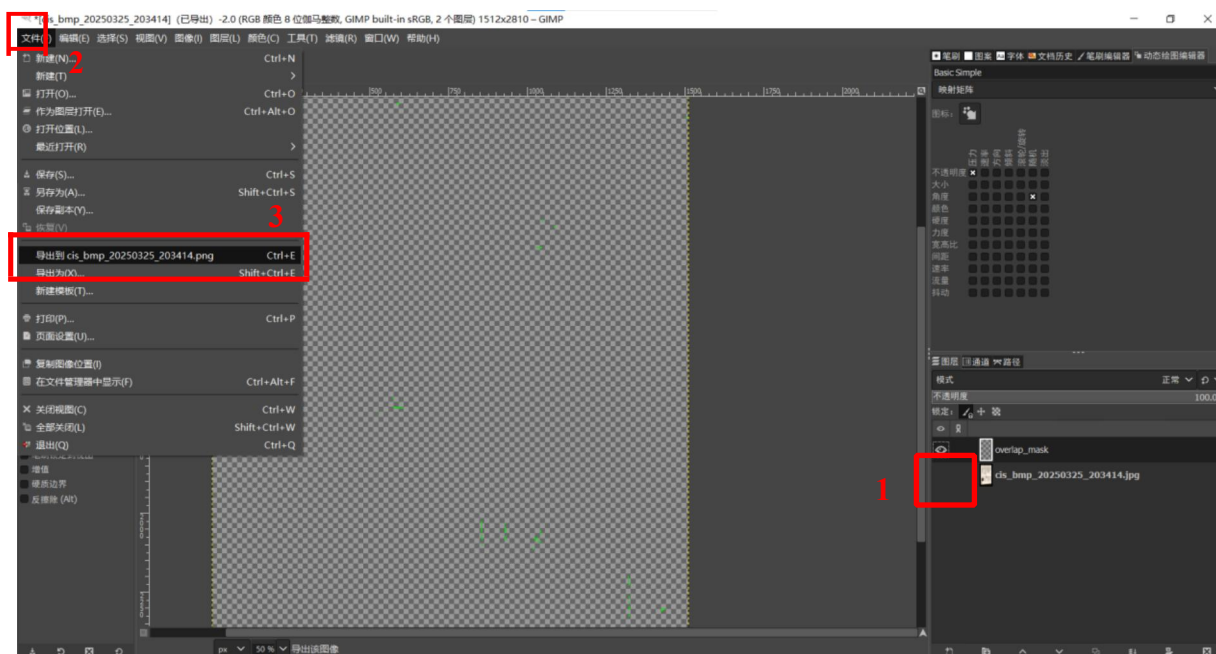


图19 导出重叠部分掩码图片 1

导出掩码图像时，将原图层隐藏，然后点击右上角导出图片

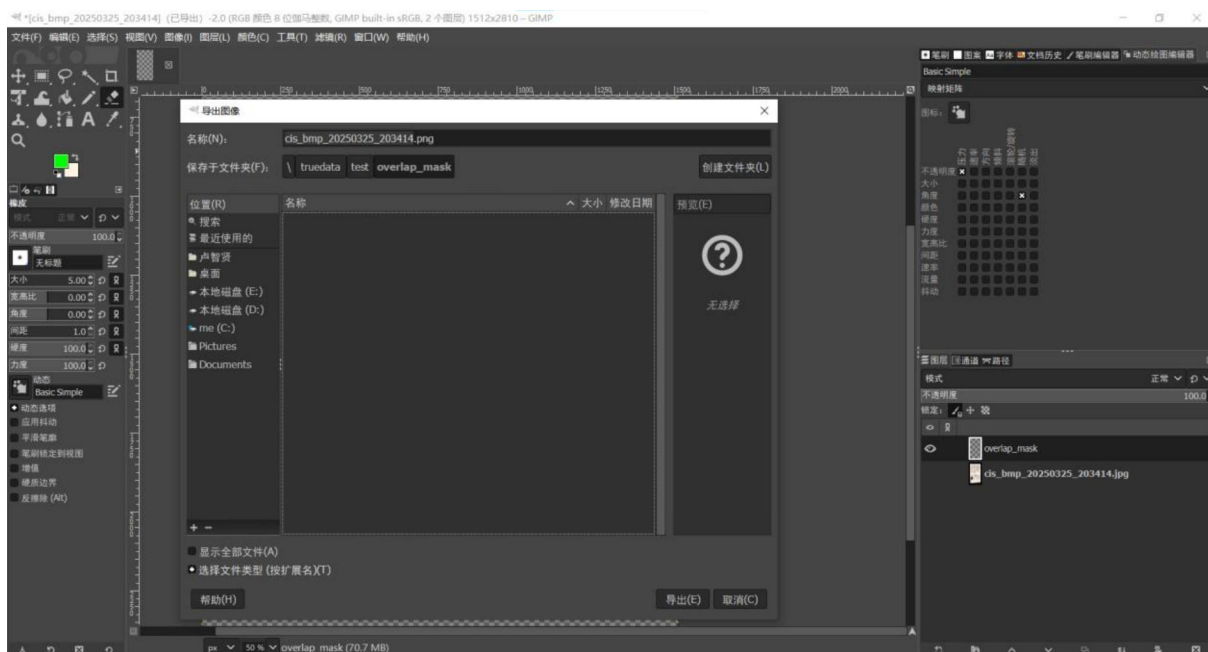


图20 导出重叠部分掩码图片 2

同样要注意导出图片的名字是否与导入时一致，文件后缀是 png，保存到 overlap_mask 目录

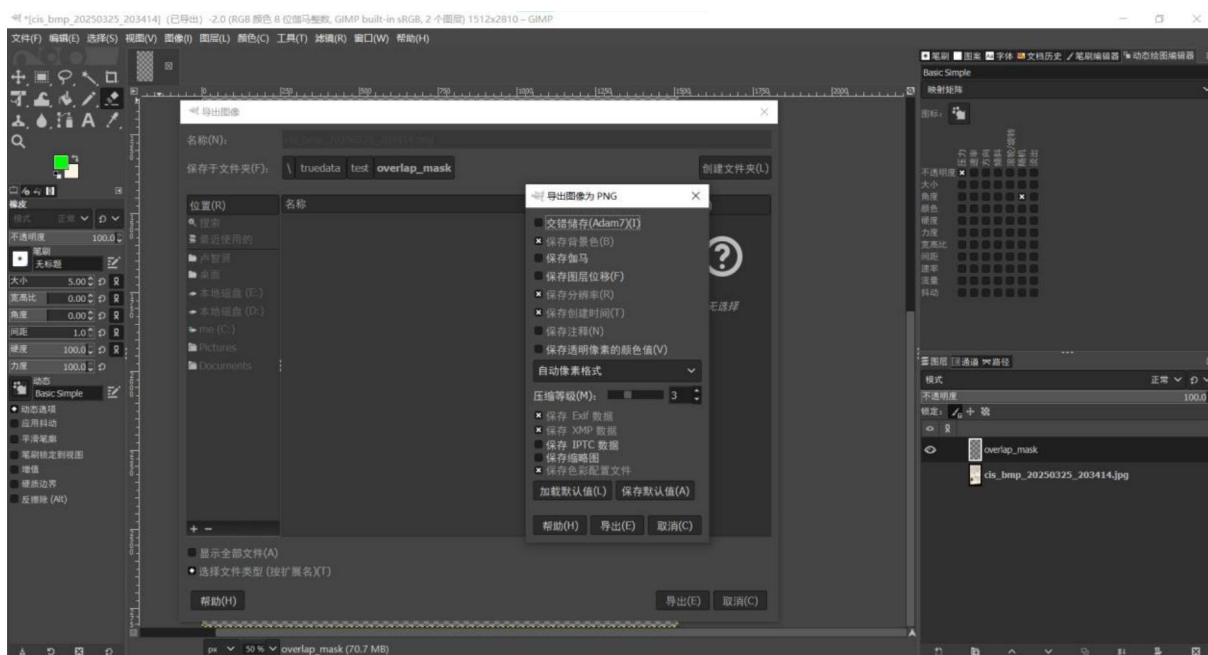


图21 导出重叠部分掩码图片 2

压缩等级选 3

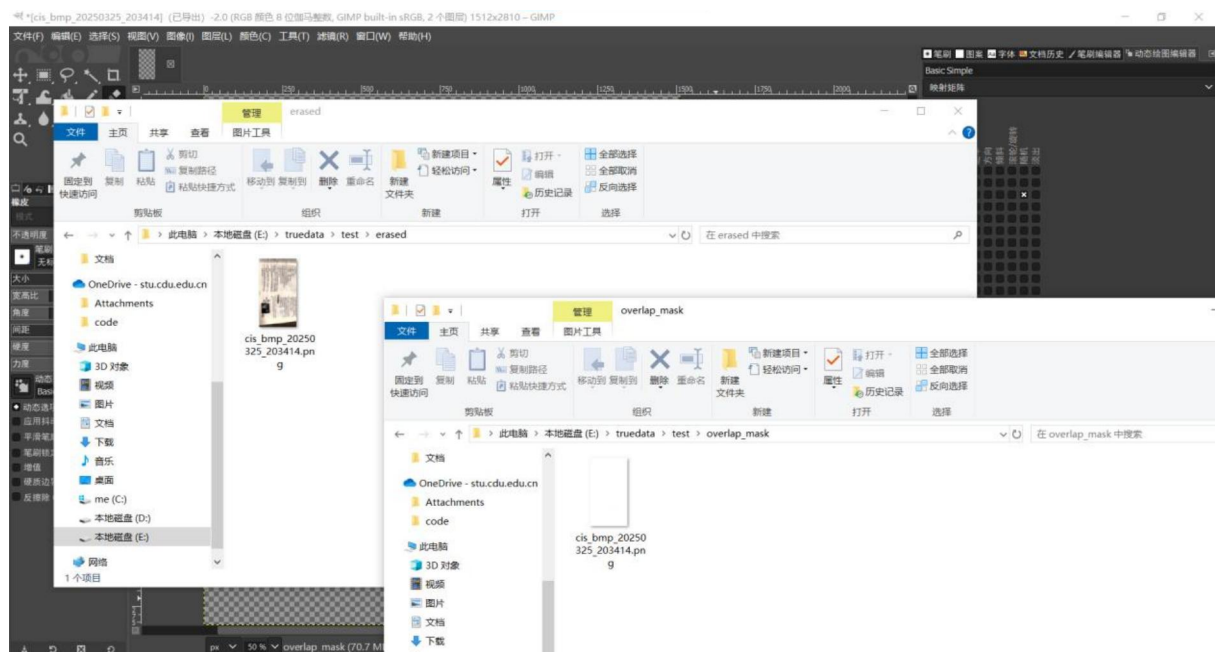


图22 导出示意图

打开目标目录，发现图片已经导出成功。

3.6 图片二值化

这一步调用脚本，对于同名的 images 图片和 erased 图片进行二值化，二值化生成的图片以同名 png 的格式保存到 images_binary 和 erased_binary 中。

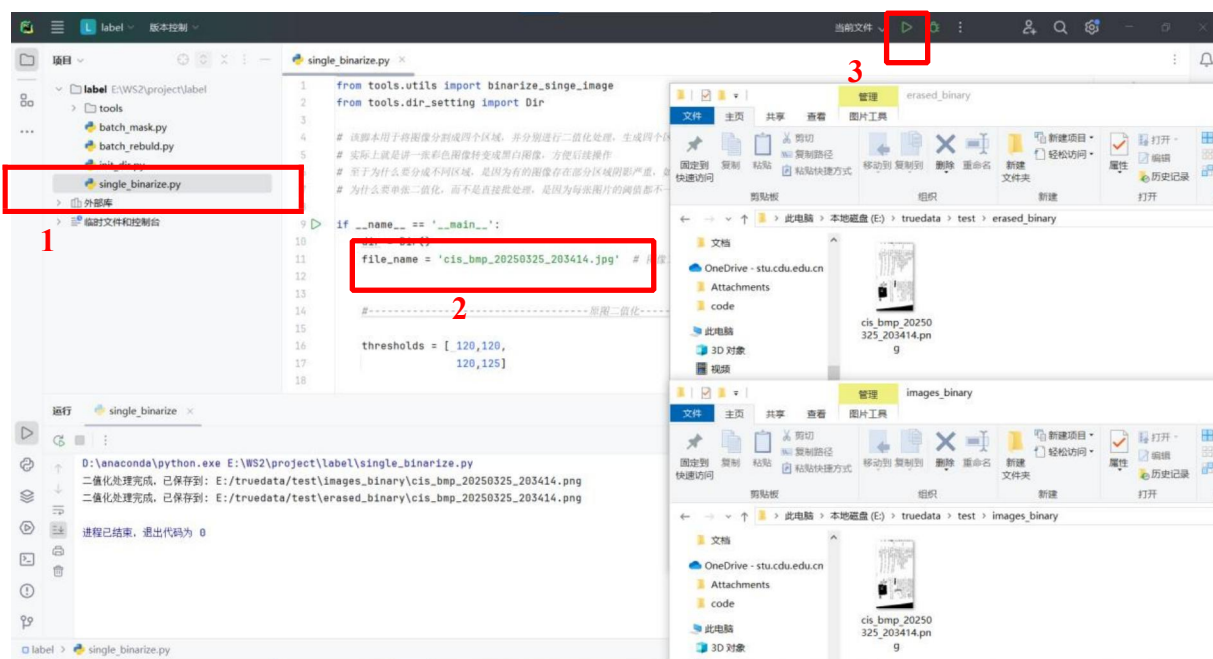


图23 二值化图片

这里使用的二值化图片只能对同一个样本的两张图片进行二值化，为什么不批量二值化呢？因为每张图片的最佳阈值都不一样，需要手动调整。为什么不对所有图像使用自适应阈值呢？因为有些图像会出现严重阴影部分，使得背景噪声和文字严重混合在一起，无法分离干净，因此需要将图片分成不同区域处理阴影部分。

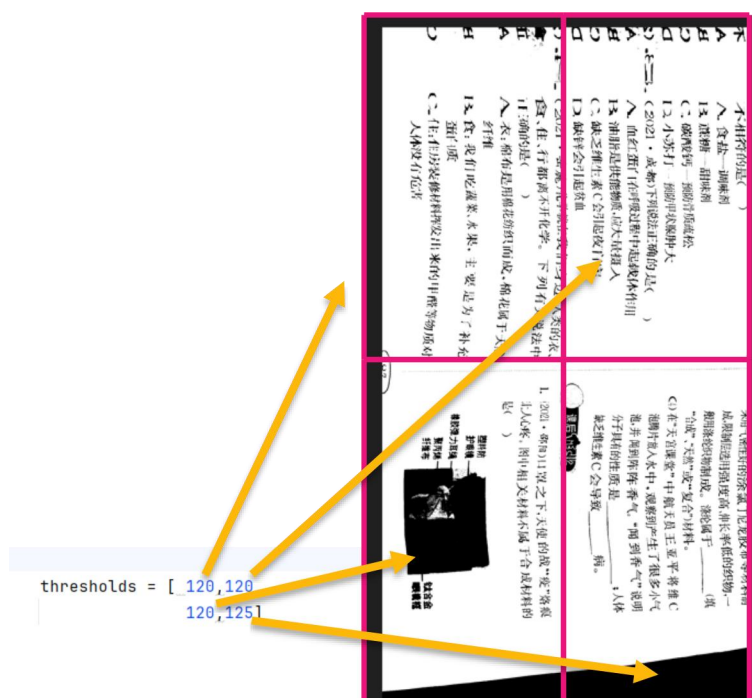


图24 阈值对应区域示意图

大多数情况下，按照这里的阈值二值化即可。阈值越低，文字保留越不完整，但噪音越小；阈值越高，文字保留越完整，但同时噪音越大。阈值建议 85-125 之间，若阴影很严重，则可以小于 85；若文字保留不完整，则可以大于 125。这里需要标注者酌情调试阈值，尽量生成效果较好的二值图。

对于背景颜色不一致的情况，可以直接使用另外一个脚本，拖动中心点划分四个域，分别对四个区域使用不同的二值化参数，可以进行二值化预览，最后选择预览效果较好的进行保存，该脚本可以替代原有的single-binarize

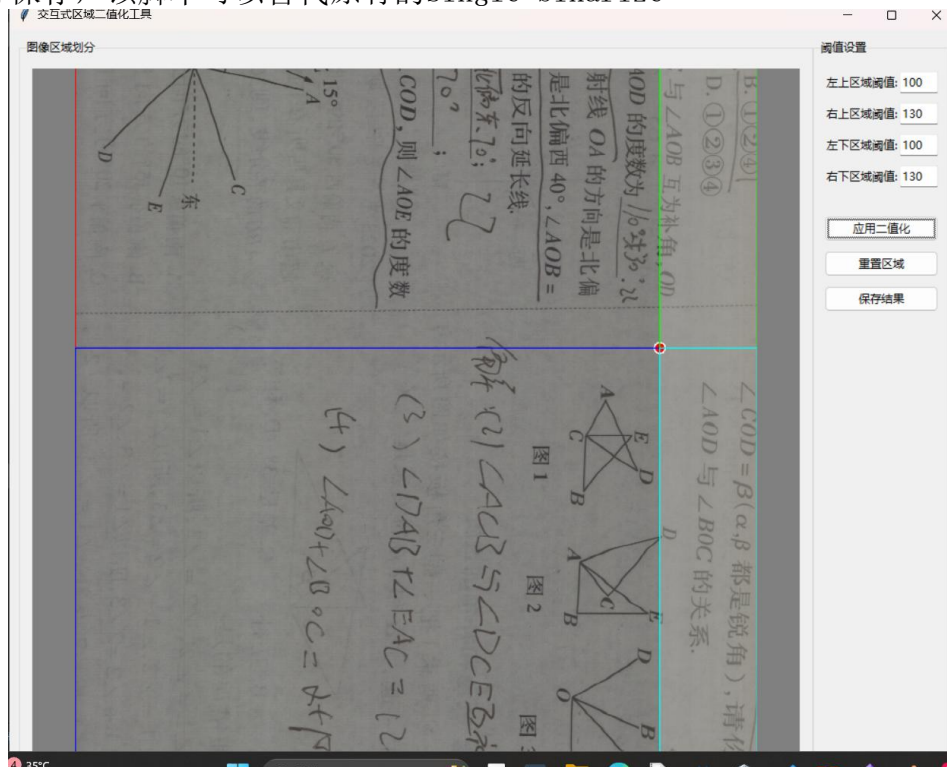


图25 图片背景颜色差异较大的情况

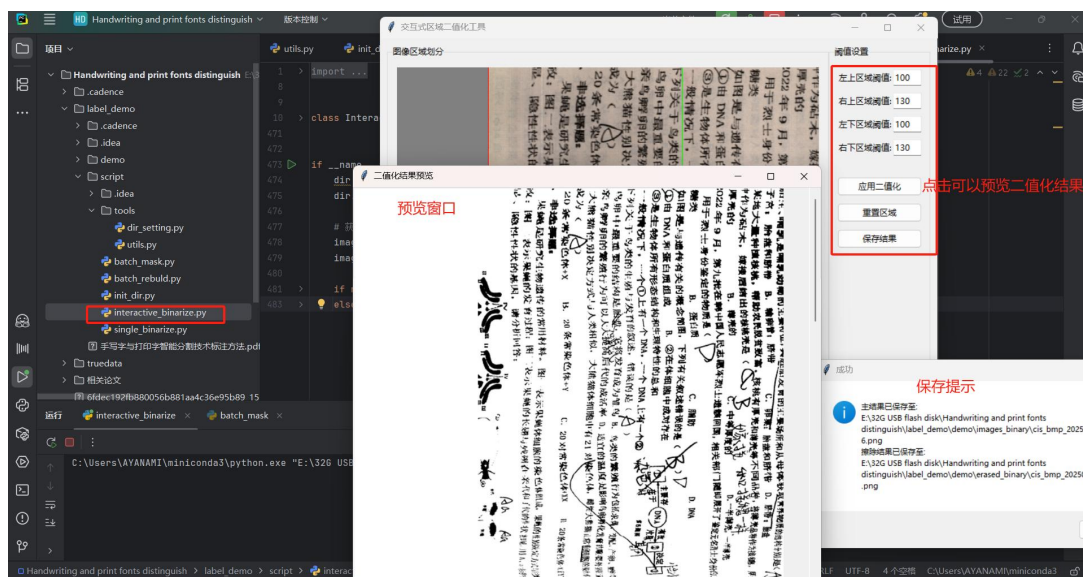


图26 二值化图片

生成的二值图中，边缘的背景（如桌面）是我们不需要的，因此将这一部分用白色擦除。

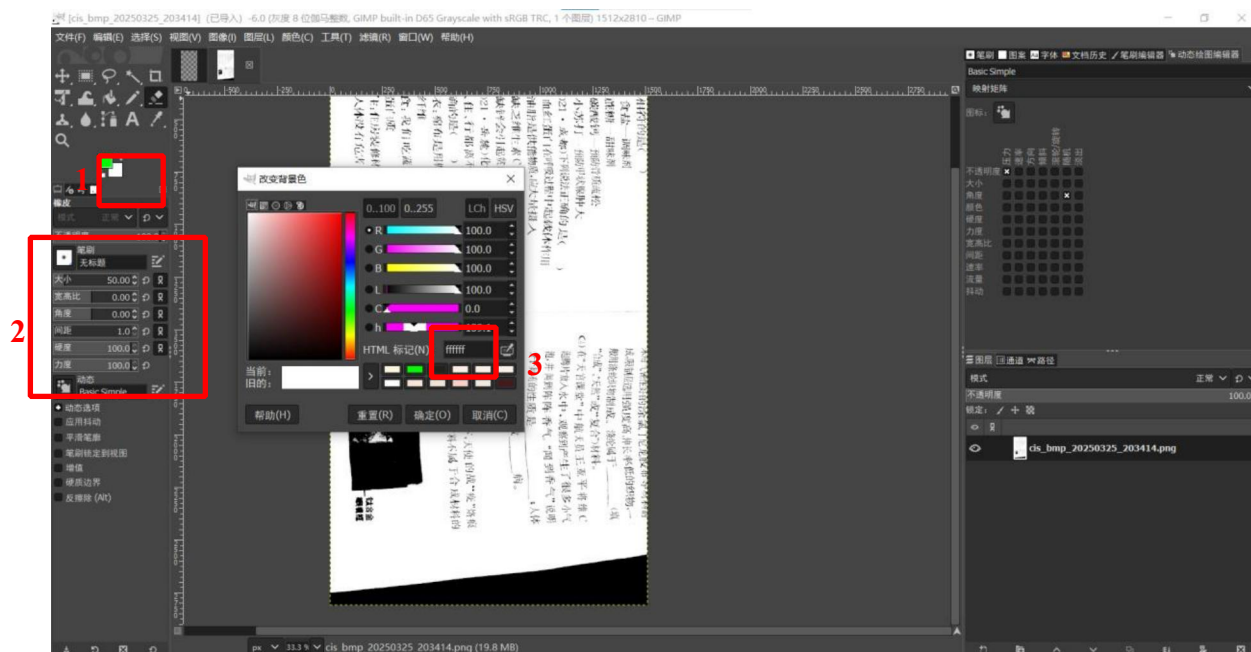


图27 擦除背景

分别导入 images_binary 和 erased_binary 中的图片，这里演示擦除 erased_binary 的背景。选择橡皮擦，颜色改成白色（FFFFFF）。橡皮擦大小可适当调大，硬度和力量设置成最大，防止擦除不均匀。

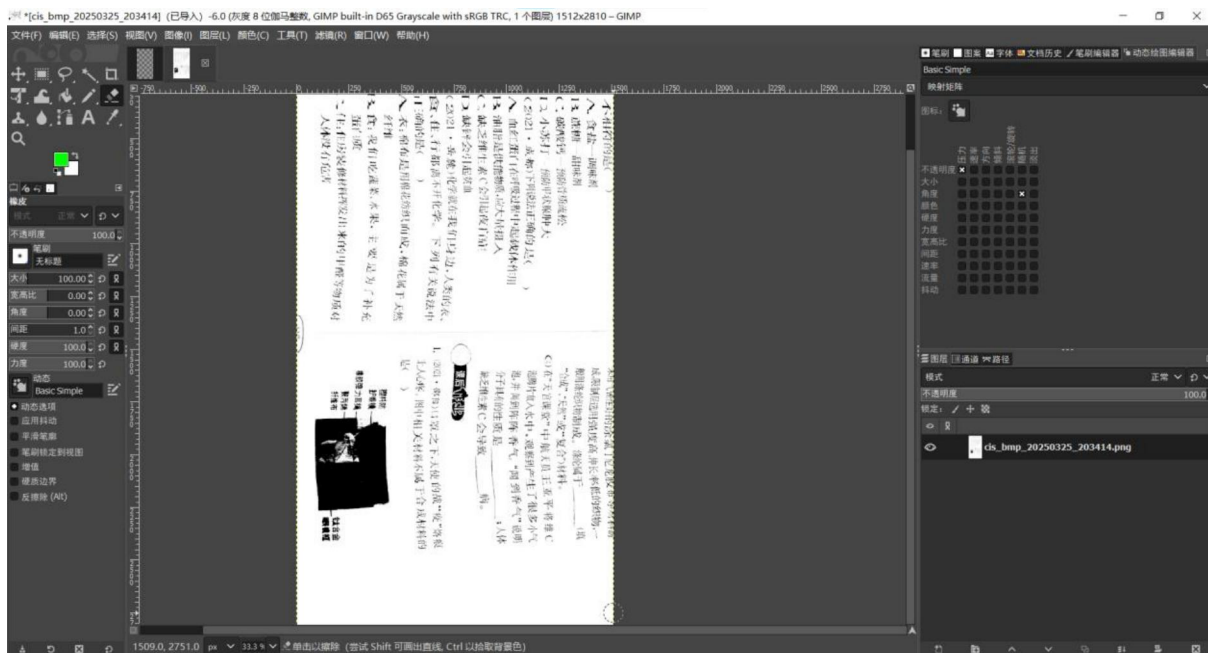


图28 擦除背景示意图

擦除以后的效果如图，主要针对边缘的黑色背景

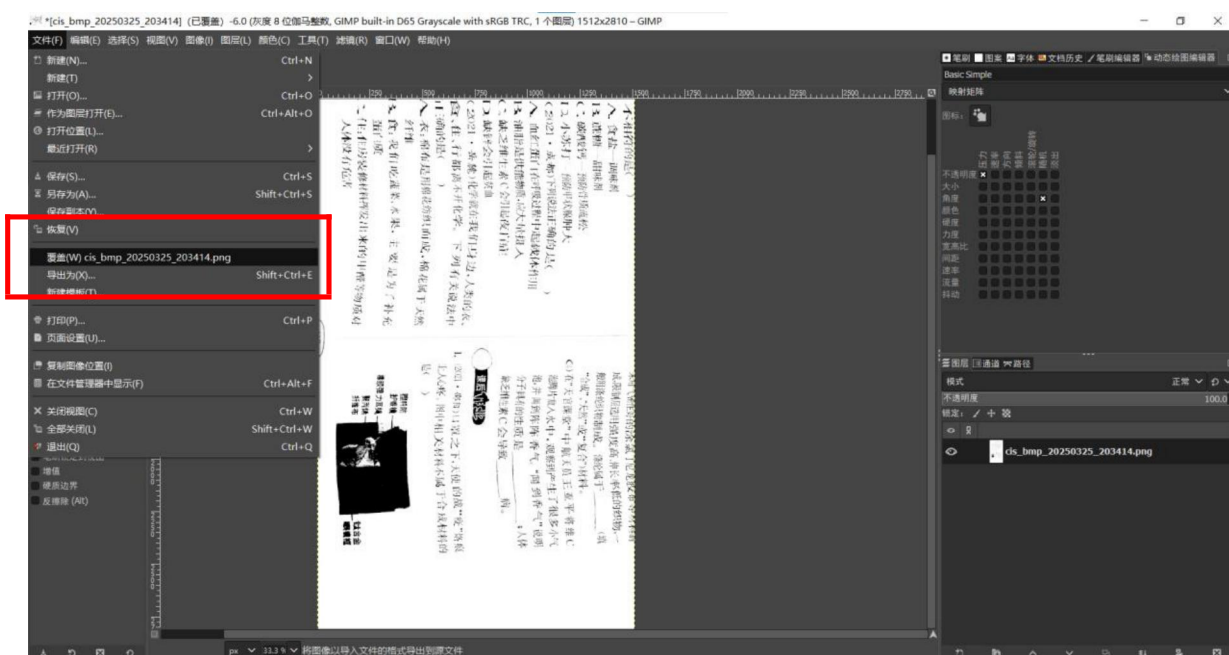


图29 导出擦除背景后的二值图

这里可以直接点覆盖原图。它会覆盖我们最近打开的一张图片。如果一次打开了多张不一样的 images，覆盖原图的时候一定要注意路径和文件名是否对应。如果担心自己弄错，推荐还是按照 3.5 的方法，通过导出为的选项导出到 erased_binary 目录中。

Images_binary 的背景擦除同理，这里不再赘述，

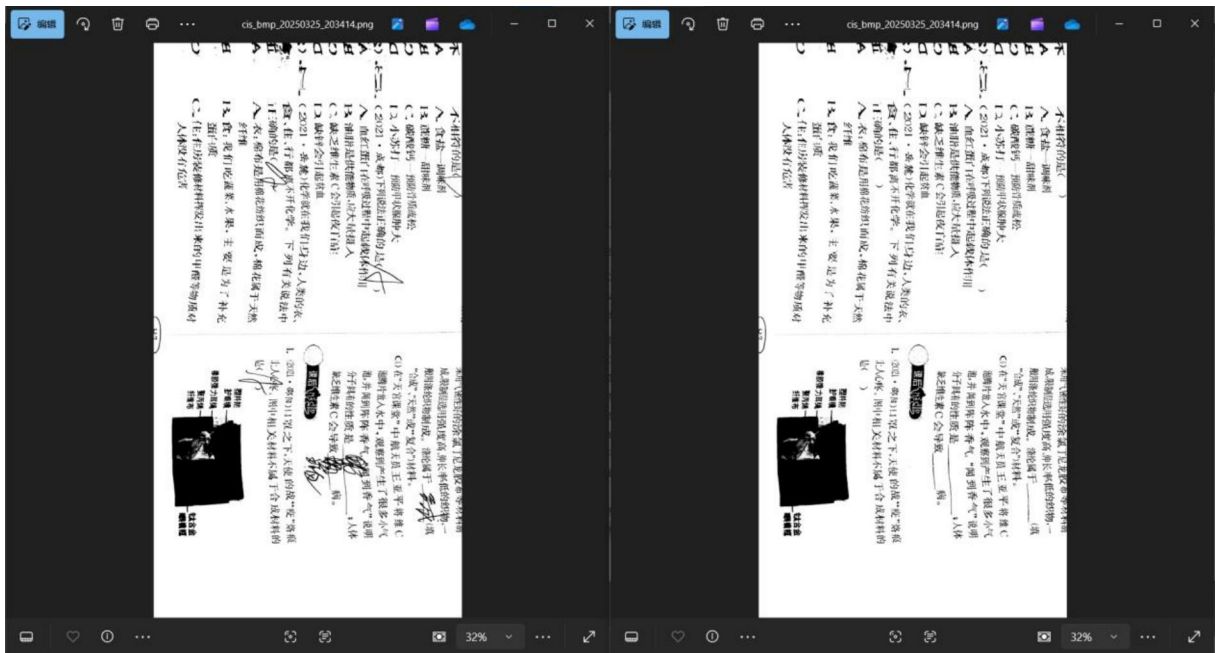


图30 对比两张二值图

最好简单地检查一下两张图片内容是否正确。首先两张二值图一定是同名的，且后缀名为 png，然后 images_binary 中的图片是同时保留了打印字（图片、表格等也算打印体）和手写字的，erased_binary 只保留打印字。

3.8 生成掩码和重构图

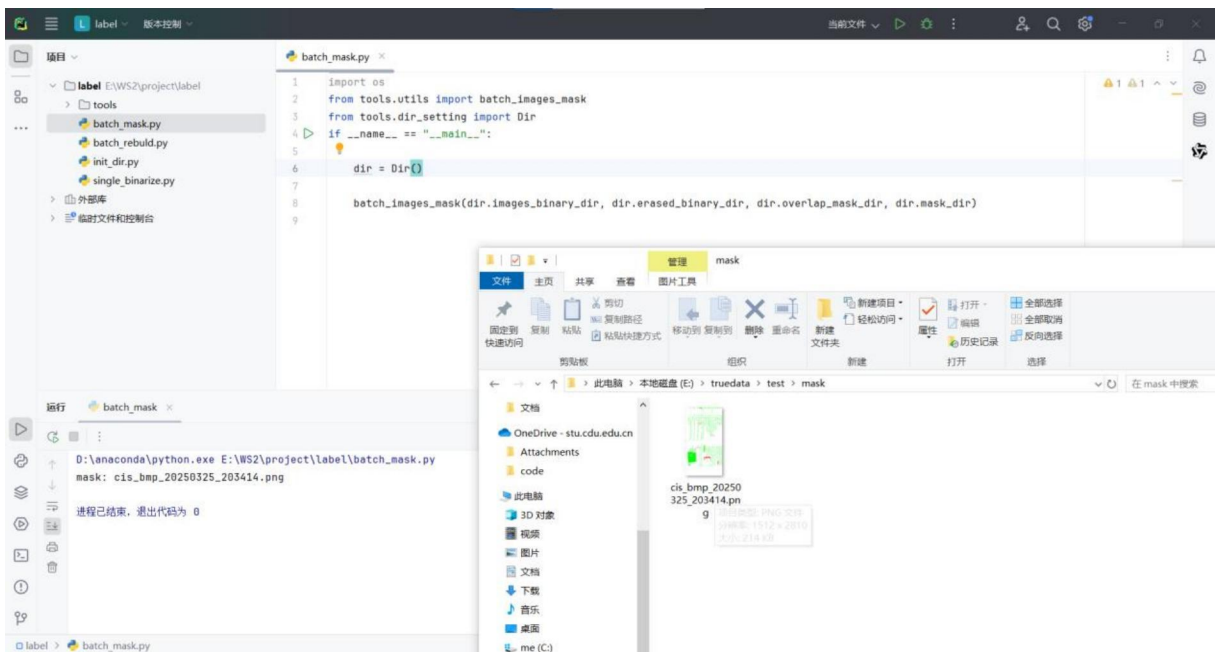


图31 掩码图片生成

调用 batch_mask.py，会对所有处理好的样本（对于同一个样本，有原图、手写字擦除图、重叠部分掩码图和两张二值图，才算是处理好）生成掩码。也就是说，如果标注者需要标注多张图片，可以等将所有样本处理好后，一次性生成所有样本的掩码图。

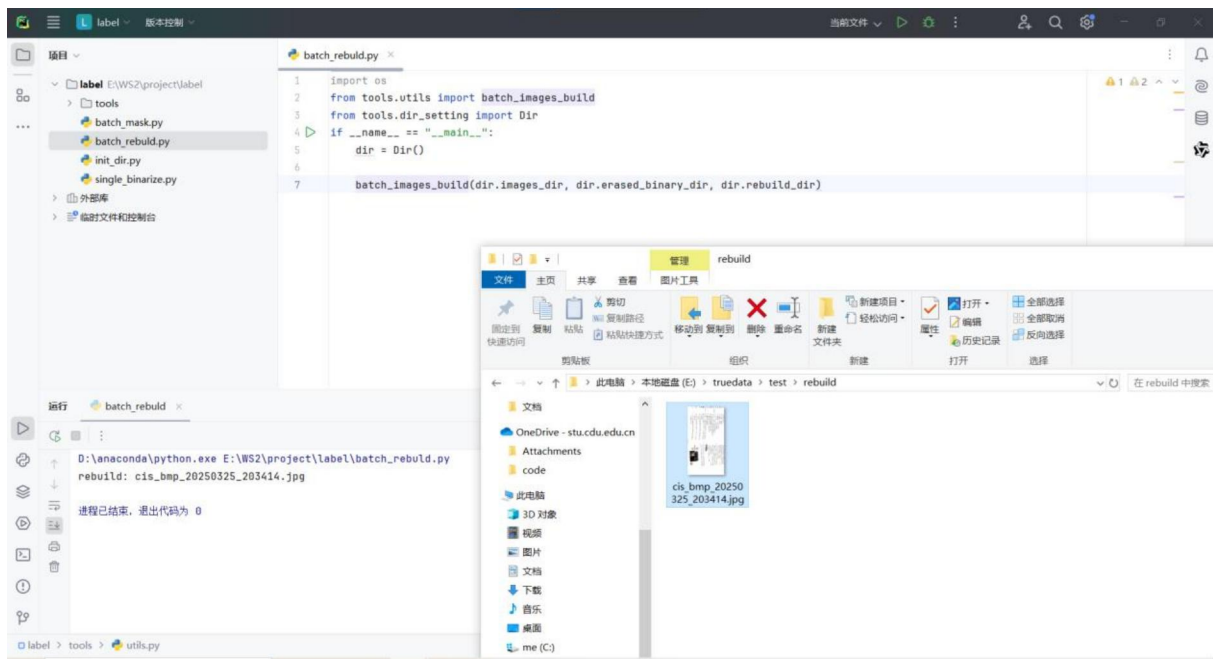


图32 GT 图生成

调用 `batch_rebuild.py`，会对所有处理好的样本生成重构图。

生成 **mask** 和 **rebuild** 图片后，可以检查一下内容是否正确。

在 **mask** 图片中，白色为背景，绿色为打印字，红色为手写字，蓝色为重叠部分像素。有轻微噪声（极少位于空白部分像素被标注成红色或者白色）可以忽略。

rebuild 图在内容分布上应当 **erased_binary** 一致，但 **rebuild** 保留的是彩色内容。

4 第二种标注方法：测试集文字掩码标注方法

与训练集的掩码标注不同，文字掩码标注无需像素级别，只需要确定单个字所在的大概位置即可

同样新建图层,words_mask，然后可以使用套索工具或者矩形框、椭圆框选择区域

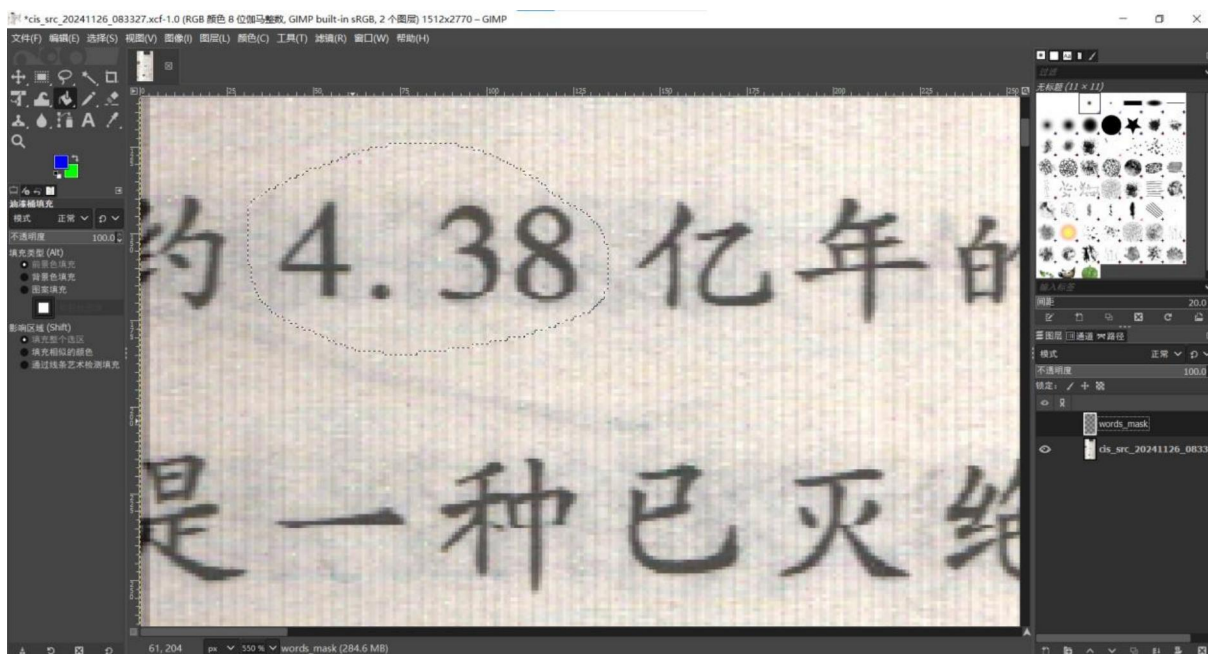


图33 选区

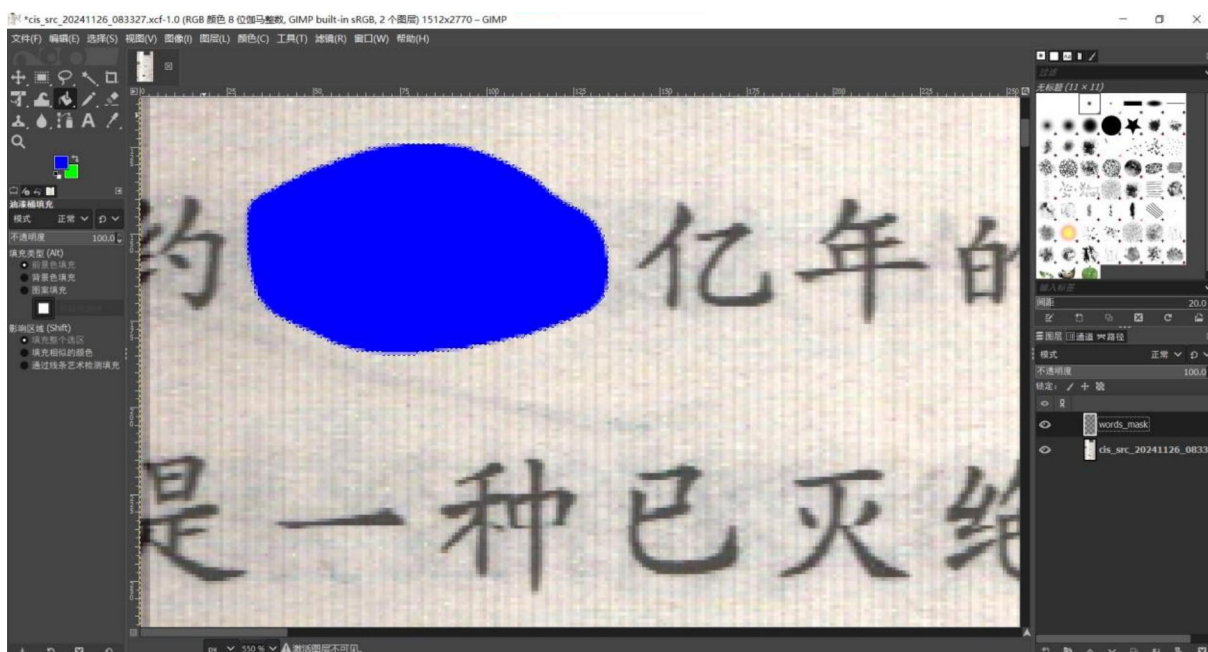


图34 填充区域

这里填充的颜色可以任意选取，**需注意，如果两个区域相邻甚至相接，一定要用不同的颜色隔开**，具体颜色任意。

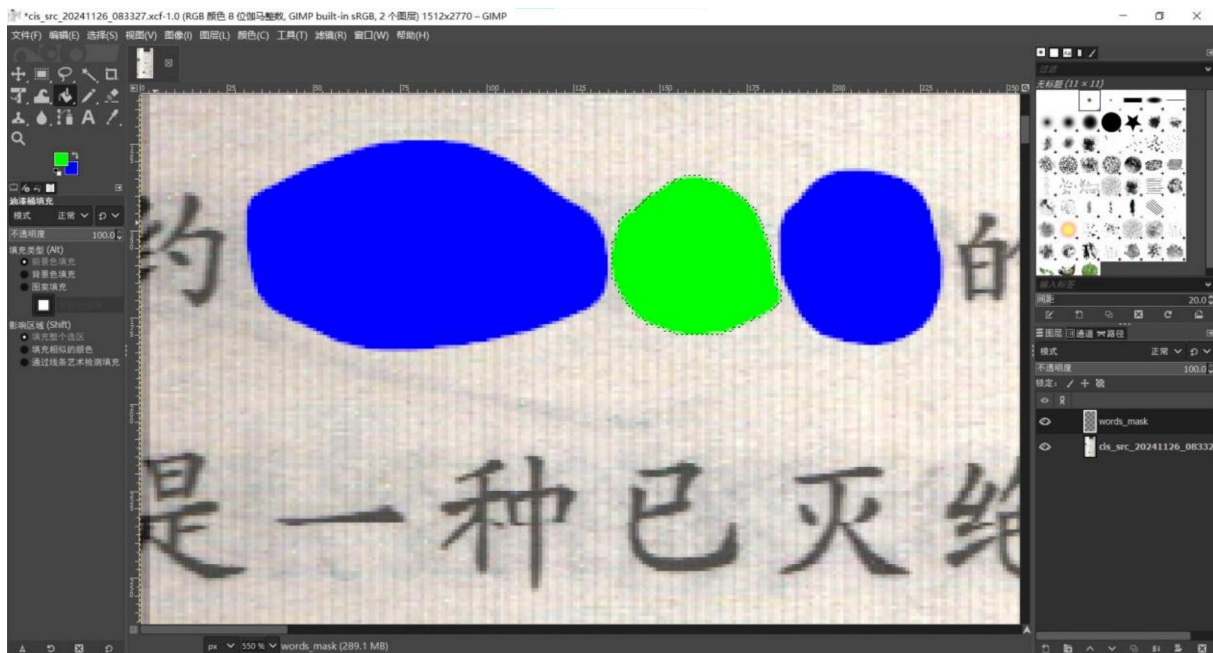


图35 效果图

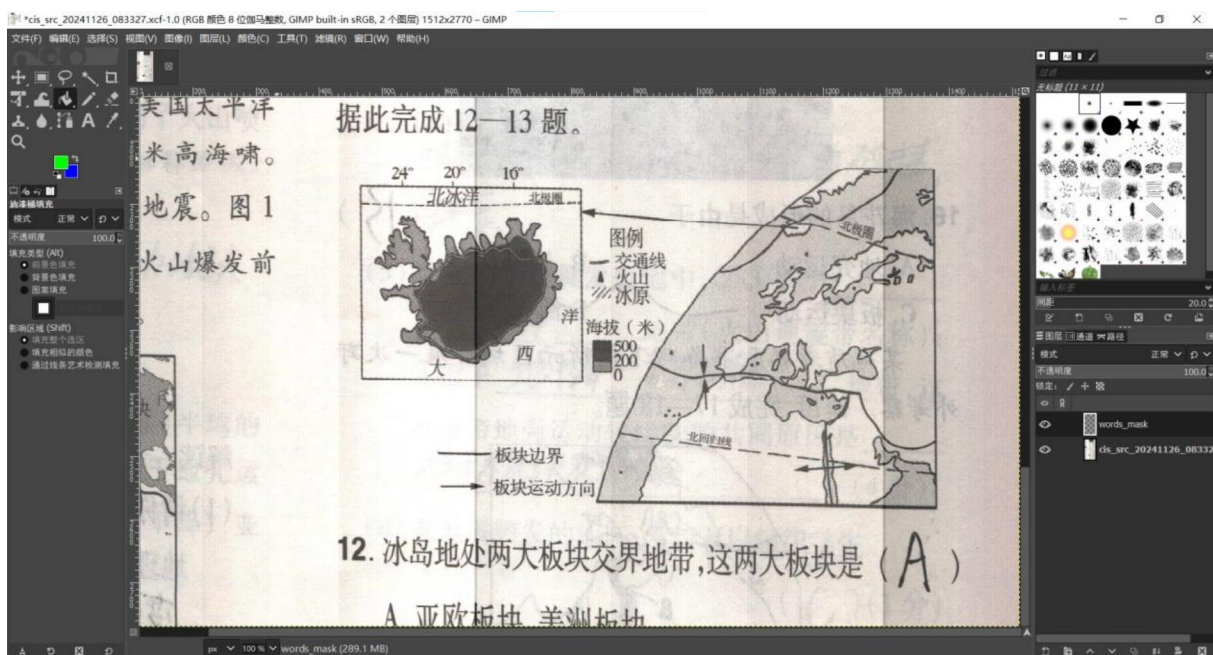


图36 图无需标注

图像部分无需花圈，标点符号也无需单独画圈

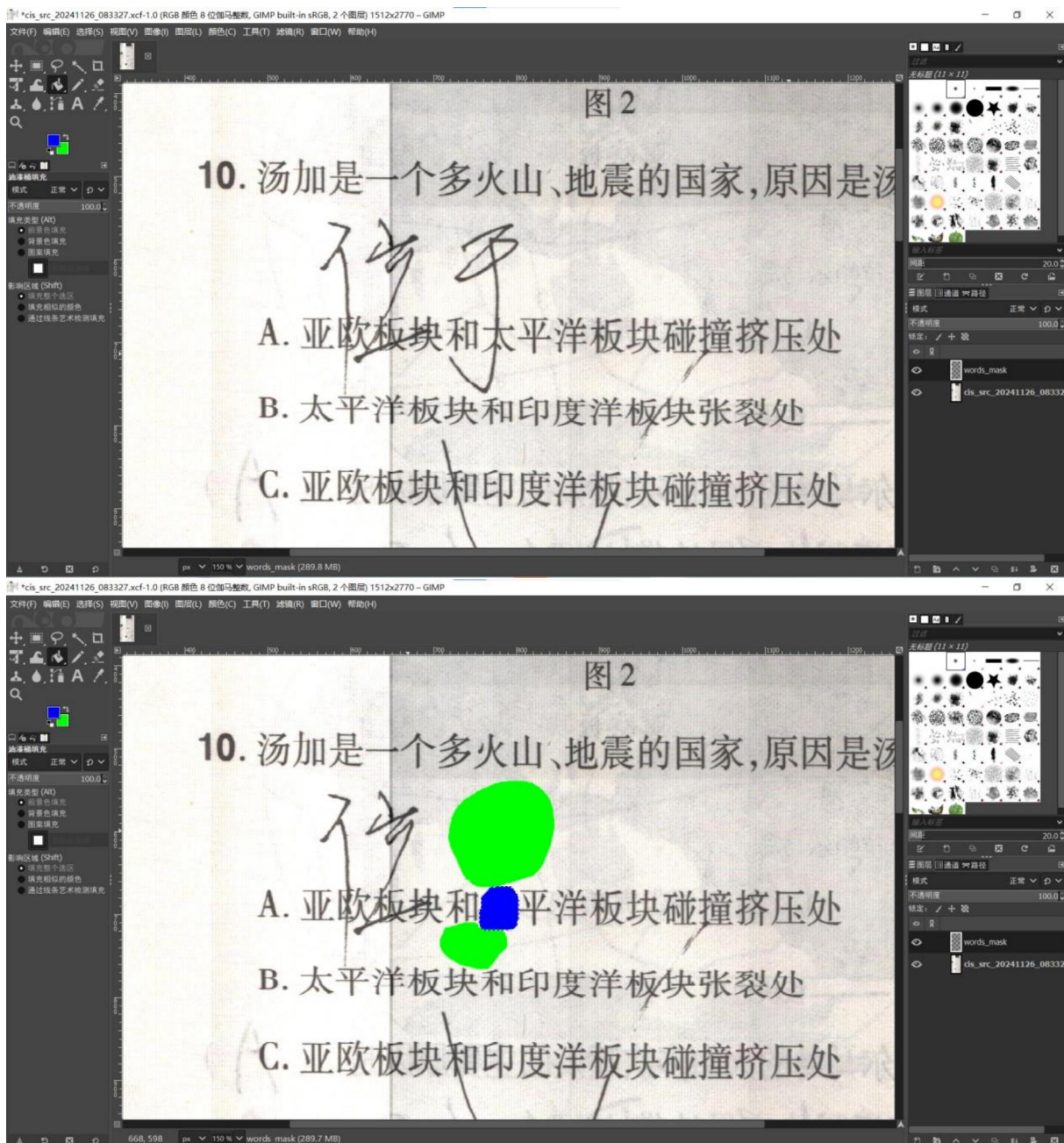


图37 重叠部分

对于可以分离的重叠部分分开画圈，难以分离的整个圈住即可，无需过于严格。

导出图像时同理，以 png 格式导出，压缩等级 3。